

ITI | April 2026

React

Islam Baidaq | Software Architect @ Vodafone

Content overview

01

JavaScript for React

02

React Fundamentals

03

State & Hooks

04

Routing & Navigation

05

Forms & Validation

06

Global State Management

JavaScript for React

Why JS matters in React

React = JavaScript + UI

JSX = JS inside UI

Arrow Functions

```
function sum(a, b) {  
  return a + b;  
}
```

```
const sum = (a, b) => a + b;
```

```
const Button = () => <button>Click</button>;
```

Template Literals

```
const name = "Islam";  
console.log(`Hello ${name}`);
```

JavaScript for React



map (CORE of React)

```
const numbers = [1, 2, 3];  
const doubled = numbers.map(n => n * 2);
```

```
users.map(user => <UserCard user={user} />)
```



Destructuring

```
const user = { name: "Islam", age: 28 };  
  
const { name, age } = user;
```



example

```
const Card = ({ title, price }) => { ... }
```


JavaScript for React

Promises

```
fetch(url)
  .then(res => res.json())
  .then(data => console.log(data));
```

Async/Await (MAIN WAY)

```
const getData = async () => {
  const res = await fetch(url);
  const data = await res.json();
};
```

Error handling

```
try {
  ...
} catch (err) {
  console.log(err);
}
```

API Data Viewer

Features:

- Fetch data from API
- Display list
- Loading state
- Error handling

Example APIs:

- <https://api.github.com/users>

React Fundamentals

1. What Problem React Solves

Before React:

- manual DOM updates
- messy code

2. Imperative vs Declarative

✗ Imperative:

```
document.getElementById("app").innerHTML = "<h1>Hello</h1>";
```

✓ Declarative:

```
const App = () => <h1>Hello</h1>;
```

React Fundamentals

3. What is React ?

React is a js library for building fast and interactive ui for web and mob

4. Virtual DOM ?

Instead of manipulating the browser's DOM directly, React creates a virtual DOM in memory, where it does all the necessary manipulating, before making the changes in the browser DOM.

5. Only changes what needs to be changed !

React finds out what changes have been made, and changes only what needs to be changed.

Setting up React (Vite)

Install : `npm create vite@latest test | npm create vite@latest my-app -- --template react-ts`

Run : `npm run dev`

◇ **Package name:**

| **testapp**

◇ **Select a framework:**

| **React**

◇ **Select a variant:**

| **JavaScript**

◇ **Install with npm and start now?**

| **Yes**

Setting up Tailwind (Vite)

Install : `npm install tailwindcss @tailwindcss/vite`

Inside `vite.config.js`

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    tailwindcss(),
  ],
})
```

Inside any Css file

```
@import "tailwindcss";
```


React Fundamentals

First Component

EX:

```
const App = () => {  
  return <h1>Hello React { version } </h1>;  
};
```

1. What is JSX?

- HTML-like syntax
- actually JavaScript

2. Rules of JSX

- one parent element
- close all tags
- use { } for JS

React Fundamentals

Components & Props

1. What is a Component?

- Reusable UI block

2. Component Types

- Functional components

3. Props (CRITICAL)

```
const Card = (props) => {  
  return <h2>{props.title}</h2>;  
};
```

4. Props Destructuring

```
const Card = ({ title, price }) => {  
  return <h2>{title}</h2>;  
};
```


React Fundamentals

Components & Props

5. Component Composition

```
<Card>  
  <button>Buy</button>  
</Card>
```

6. Children Prop

```
const Card = ({ children }) => {  
  return <div>{children}</div>;  
};
```

React Fundamentals

Rendering & Re-rendering

1. What is Rendering?

Explain:

- React renders UI → based on state/props

2. What causes re-render?

- state change
- props change
- parent re-render

```
const App = () => {  
  console.log("render");  
  return <h1>Hello</h1>;  
};
```


React Fundamentals

Events in React

1. Handling Events

```
<button onClick={() => console.log("clicked")}>  
  Click  
</button>
```

2. Passing Functions

```
const handleClick = () => { ... };  
  
<button onClick={handleClick} />
```

2. Event Object

```
const handleClick = (e) => {  
  console.log(e.target);  
};
```

```
<input onChange={(e) => console.log(e.target.value)} />
```

React Fundamentals

Project: Product List UI

Requirements:

- Display list of products
- Each product in a Card component
- Pass props (title, price, image)
- Render list using `.map()`
- Add button click (log action)

State & Hooks

State

- State store property values that belongs to the component itself, when the state changes, the component re-render.
- State managed within the component .
- State can be changed

👉 State = data that changes UI

👉 “What happens when `setCount` runs?”

Answer:

- **component function runs again**
- **UI updates**

```
const Counter = () => {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      {count}  
    </button>  
  );  
};
```

State & Hooks

1. State is immutable

Immutable state means never modifying the existing state directly; always create a new object or array when updating it. { React usually compares by reference, not deep value. }

 Wrong:

```
state.push(item);
```

 Correct:

```
setItems([...items, item]);
```

2. Multiple State

```
const [name, setName] = useState("");
```

```
const [age, setAge] = useState(0);
```


State & Hooks

1. useEffect Basics

👉 Side effects = things outside rendering

Examples:

- API calls
- timers
- subscriptions

2. Basic useEffect

```
useEffect(() => {  
  console.log("component mounted");  
}, []);
```

3. Dependency Array

```
useEffect(() => {  
  console.log(count);  
}, [count]);
```

Dependency
When it runs

State & Hooks

useEffect Dependency Array

Dependency

[]

When it runs

once

[count]

when count changes

(no array)

every render

Advanced Hooks

2. useMemo

lets React **remember a calculated value** and reuse it on the next renders **until its dependencies change**.

{ cache the result of a function }

```
const value = useMemo(() => compute(), [deps]);
```

3. useCallback

useCallback lets React **remember a function reference** between renders.

{ cache the function itself }

```
const fn = useCallback(() => {}, [deps]);
```

Advanced Hooks

Project: Advanced Todo App

Features:

- add task
- delete task
- toggle complete
- filter (all / completed)
- persist in localStorage
- useEffect for saving

Routing & Navigation

Routing Fundamentals

1. What is Routing?

Explain:

- Traditional websites → multiple pages (server)
- React → **Single Page App (SPA)**

👉 Routing = changing UI based on URL

2. Install React Router

```
npm install react-router-dom
```

Routing & Navigation

Routing Fundamentals

3. Basic Setup

```
import { BrowserRouter, Routes, Route } from "react-router-dom";  
  
<BrowserRouter>  
  <Routes>  
    <Route path="/" element={<Home />} />  
    <Route path="/about" element={<About />} />  
  </Routes>  
</BrowserRouter>
```

4. Navigation with Link

```
import { Link } from "react-router-dom";  
  
<Link to="/about">Go to About</Link>
```


Routing & Navigation

Dynamic Routing

1. Route Parameters

```
<Route path="/user/:id" element={<User />} />
```

2. Access Params

```
import { useParams } from "react-router-dom";  
  
const { id } = useParams();
```

Routing & Navigation

Extend Previous App (Task Manager)

Add:

- multiple pages:
 - Home
 - Tasks
 - Profile
- navigation bar
- dynamic route:
 - `/task/:id`